

Software Tools and Techniques for CSE: Assessment 2

Project Report

Ridham Patel (23110238), Shardul Junagade (23110297)

19 October 2025

Computer Science & Engineering
Indian Institute of Technology Gandhinagar
India

Repository Link: `cs202-stt`

Contents

1	Lab Assignment 6	2
1.1	Project Selection	2
1.2	Tool Selection	2
1.3	Pairwise Findings of Vulnerability Tools on Project	3
1.4	Tool-level CWE Coverage Analysis	4
1.5	Pairwise Agreement (IoU) Analysis	5
1.5.1	Interpreting the IoU Matrix	6
1.5.2	Which tool combination maximizes CWE coverage?	6
1.6	References	6
2	Lab Assignment 7: Reaching Definitions Analyzer for C Programs	7
2.1	Introduction, Setup, and Tools	7
2.1.1	Introduction	7
2.1.2	Environment and Tools	7
2.2	Program Corpus	7
2.3	Approach and Implementation	8
2.3.1	Parsing and Cleaning – <code>analyzer/c_parser.py</code>	8
2.3.2	Basic Blocks and CFG Construction – <code>analyzer/cfg_builder.py</code>	9
2.3.3	Definition Extraction – <code>analyzer/cfg_builder.py</code>	11
2.3.4	gen/kill Sets per Block – <code>analyzer/cfg_builder.py</code>	12
2.3.5	Reaching Definitions (Data-Flow) – <code>analyzer/dataflow.py</code>	13
2.3.6	Ambiguity Detection – <code>analyzer/dataflow.py</code>	14
2.3.7	Runner and Outputs	15
2.4	Results	16
2.4.1	CFG and Metrics	16
2.4.2	Reaching Definitions Tables	17
2.4.3	Ambiguity Highlights	17
2.5	Discussion and Reflection	17
2.5.1	Challenges	17
2.5.2	What I Learned	17
2.5.3	Further Improvements	17
2.6	References	17

1 Lab Assignment 6

Objective: The objective is to assess how different vulnerability analysis tools detect software weaknesses categorized by CWE, compute tool-level coverage including the CWE Top 25 set, and analyze pairwise similarity/diversity using the Jaccard Index (IoU).

Learning Outcomes: By the end of this lab:

1. Apply multiple vulnerability analysis tools to real-world projects.
2. Extract CWE-based findings from tool outputs.
3. Compare tools by CWE coverage (breadth of vulnerabilities detected).
4. Compute and interpret pairwise IoU values between tools.
5. Visualize and analyze CWE detection results.

Pre-Lab Requirements: Environment: MacOS with Python 3.13.3. Prior reading: “Comparison and Evaluation on Static Application Security Testing Tools (FSE 2023).”

1.1 Project Selection

We selected repositories that are active and non-trivial in size so the SAST tools would exercise a variety of code paths. Selection metrics used: We used Stars and forks for popularity signals from GitHub. We set the threshold for stars and forks as 150 and 50 respectively. We used the number of Commits as a metric to measure development activity (number of commits on default branch). We set the commit threshold to be 1500.

We used Github Sear tool to filter and search repositories. Best found results as reported below were taken as selected repositories.

Project	GitHub (short)	Stars	Forks	Commits
dl-art-school	github.com/152334h/dl-art-school	188	54	2.2k
instructor	github.com/567-labs/instructor	11.6k	869	1.5k
two1-python	github.com/21dotco/two1-python	366	90	1.9k

Table 1: Selected projects and selection metrics. ”Stars/Forks/Commits” indicate popularity and activity.

1.2 Tool Selection

We select three SAST tools that explicitly support CWE-tagged reporting (directly or via a reproducible mapping step).

Selected Tools

1. **Semgrep:** Invocation:

```
semgrep --config <rules.yaml or registry:rules> --json -o
raw_tool_outputs/<project>__semgrep.json <project_path>
```

Version check: `semgrep --version`. CWE support: Semgrep rules may include CWE identifiers in rule metadata (e.g., `metadata.cwe`); the Semgrep JSON output exposes this under `extra.metadata` and is parsed by the aggregation pipeline.

2. **Dlint:** Invocation (via flake8/dlint):

```
# using flake8 with dlint enabled (JSON output)
flake8 --select=DL --format=json <project_path> >
raw_tool_outputs/<project>__dlint.json
```

Version check: `pip show dlint or flake8 --version`. CWE support: Dlint rules do not always embed CWE identifiers natively; the pipeline applies a reproducible mapping (rule code → CWE) so findings are normalized to CWE IDs for coverage and IoU calculations.

3. **Bandit:** Invocation:

```
bandit -r -f json -o raw_tool_outputs/<project>__bandit.json <project_path>
```

Version check: `bandit --version`. CWE support: Bandit JSON frequently includes CWE information (or can be mapped from `test_id` to CWE). The aggregator extracts the CWE field when present and falls back to a maintained `test_id`→CWE mapping when necessary.

1.3 Pairwise Findings of Vulnerability Tools on Project

To find pairwise vulnerability and document it, we run all the tools on all the datasets and performed the following data processing:

1. Aggregate unique CWE IDs per tool per project and overall.
2. Compute Top 25 coverage as a percentage.
3. Derive pairwise IoU using Jaccard Index over CWE ID sets.

Processing and QC: After each tool run we enforce:

1. Structured export: JSON/SARIF/CSV for deterministic parsing.
2. CWE normalization: canonicalize identifiers to the form `CWE-<n>` (strip leading zeros, unify case).
3. Exclude third-party vendored directories and test-only folders to reduce noise (project-specific exclude lists applied where appropriate).

Algorithm 1 Parse and normalize tool outputs

Require: `raw_tool_outputs` directory, `TOP25_SET`

```

1: aggregated ← empty map keyed by (project, tool, cwe) → count
2: for all file in raw_tool_outputs do
3:   (project, tool) ← parse filename(file)
4:   findings ← parse_tool_output(file) {tool-specific parser}
5:   for all finding ∈ findings do
6:     cwe ← normalize_cwe(finding.cwe) {e.g. "CWE-022" to "CWE-22"}
7:     if cwe is not None then
8:       aggregated[(project,tool,cwe)] += 1
9:     end if
10:  end for
11: end for
12: return aggregated
```

For each project-tool pair, scans are executed and findings exported with CWE IDs, aggregated, and consolidated into a single CSV using the following pseudocode:

Algorithm 2 Write consolidated CSV

Require: aggregated (from previous), `TOP25_SET`

```

1: open CSV with header (Project_name,Tool_name,CWE_ID,Number_of_Findings,Is_In_CWE_Top_25?)
2: for all (project,tool,cwe) → count in aggregated do
3:   top25_flag ← (cwe ∈ TOP25_SET) ? "Yes" : "No"
4:   write row (project,tool,cwe,count,top25_flag) to CSV
5: end for
```

Shown below is the output of the code when run over all the datasets:

Table 2: Consolidated CWE findings (excerpt)

Project_name	Tool_name	CWE_ID	Number_of_Findings	Is_In_CWE_Top_25?
dl-art-school	bandit	CWE-338	153	No
dl-art-school	bandit	CWE-428	6	No
dl-art-school	bandit	CWE-502	4	Yes
dl-art-school	bandit	CWE-78	14	Yes
dl-art-school	dlint	CWE-502	4	Yes
dl-art-school	dlint	CWE-78	4	Yes
dl-art-school	semgrep	CWE-22	34	Yes
dl-art-school	semgrep	CWE-502	5	Yes
dl-art-school	semgrep	CWE-78	3	Yes
instructor	bandit	CWE-338	8	No
instructor	bandit	CWE-428	1	No
instructor	bandit	CWE-78	1	Yes
instructor	bandit	CWE-89	1	Yes
instructor	semgrep	CWE-22	22	Yes
two1-python	bandit	CWE-338	13	No
two1-python	bandit	CWE-428	75	No
two1-python	bandit	CWE-78	82	Yes
two1-python	bandit	CWE-89	3	Yes
two1-python	dlint	CWE-78	11	Yes
two1-python	semgrep	CWE-22	31	Yes
two1-python	semgrep	CWE-502	5	Yes
two1-python	semgrep	CWE-78	4	Yes

1.4 Tool-level CWE Coverage Analysis

We compute two complementary Top-25 coverage metrics for each tool (see algorithm 3):

1. Top25_Over_25 (%): fraction of the official Top-25 CWE categories covered by the tool ($\frac{\text{Top25_covered}}{25} \times 100$).
2. Top25_Fraction_Of_Tool (%): fraction of the tool's unique CWE set that is in the Top-25 ($\frac{\text{Top25_covered}}{\text{Unique CWEs}} \times 100$).

Algorithm 3 Compute per-tool sets and Top-25 metrics

Require: consolidated CSV, TOP25_SET

```

1: tool_sets  $\leftarrow$  map tool  $\rightarrow$  set()
2: for all row in CSV do
3:   tool_sets[row.tool].add(row.CWE_ID)
4: end for
5: for all tool, set_cwes  $\in$  tool_sets do
6:   unique_count  $\leftarrow$  —set_cwes—
7:   top25_covered  $\leftarrow$  —set_cwes  $\cap$  TOP25_SET—
8:   pct_over_25  $\leftarrow$  top25_covered / 25.0 * 100
9:   pct_fraction  $\leftarrow$  (top25_covered / unique_count * 100) if unique_count  $\neq$  0 else 0
10:  emit (tool, unique_count, top25_covered, pct_over_25, pct_fraction)
11: end for

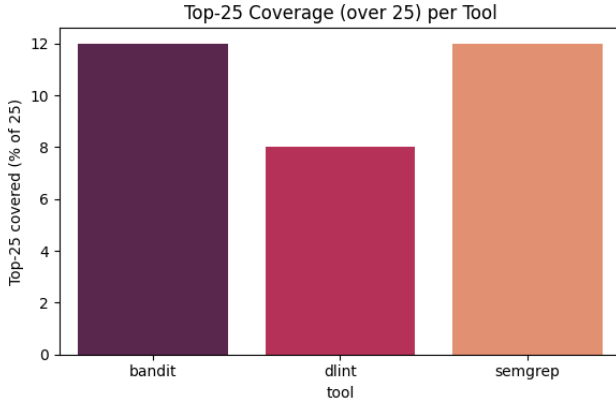
```

Shown below is the Table of coverage analysis implemented as stated above:

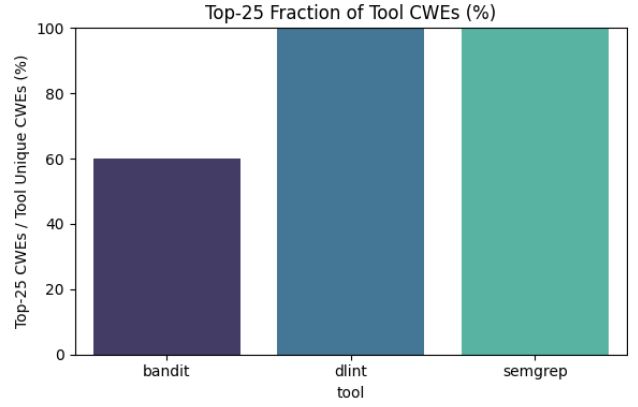
Tool	Unique CWEs	Top25 Covered	Top25 over 25 (%)	Top25 fraction of tool (%)
bandit	5	3	12.0	60.0
dlint	2	2	8.0	100.0
semgrep	3	3	12.0	100.0

Table 3: Tool-level CWE coverage summary.

Coverage can be visually analyzed using plots given in figure 1a & 1b.



(a) Top-25 coverage (over 25) per tool



(b) Top-25 fraction of tool CWEs (%)

1.5 Pairwise Agreement (IoU) Analysis

IoU Definition: For tool sets S_{T_1} and S_{T_2} of unique CWE IDs, the IoU is:

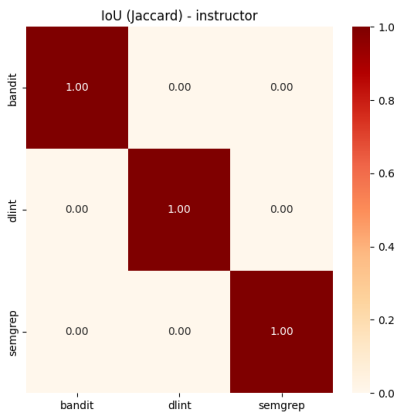
$$\text{IoU}(T_1, T_2) = \frac{|S_{T_1} \cap S_{T_2}|}{|S_{T_1} \cup S_{T_2}|}.$$

This is equivalent to the Jaccard Index on sets of detected CWE categories.

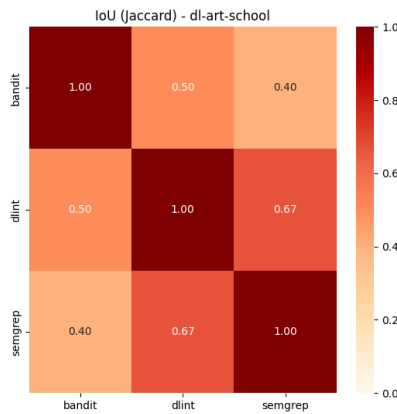
Algorithm 4 Compute pairwise IoU (tool-level and per-project)

Require: consolidated CSV, OPTIONAL REPORT_TOOLS list

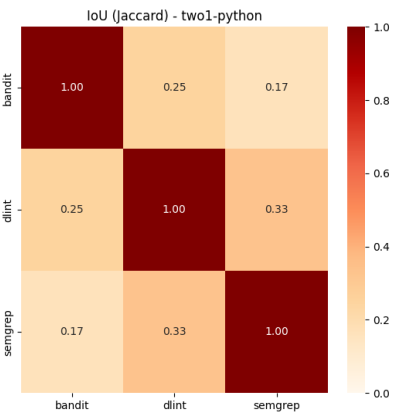
- 1: tools $\leftarrow$ REPORT_TOOLS if set else sorted(all tools in CSV)
 - 2: tool_sets $\leftarrow$ (for each tool: union of CWEs across projects)
 - 3: IoU_matrix $\leftarrow$ matrix(len(tools), len(tools))
 - 4: **for** i=1 to len(tools) **do**
 - 5: **for** j=1 to len(tools) **do**
 - 6: A $\leftarrow$ tool_sets[tools[i]]
 - 7: B $\leftarrow$ tool_sets[tools[j]]
 - 8: **if** A empty and B empty **then**
 - 9: IoU_matrix[i,j] $\leftarrow$ 1.0
 - 10: **else**
 - 11: IoU_matrix[i,j] $\leftarrow$ $\frac{|A \cap B|}{|A \cup B|}$
 - 12: **end if**
 - 13: **end for**
 - 14: **end for**
 - 15: {Repeat same per-project by building tool sets restricted to each project}
-



(a) Instructor: IoU

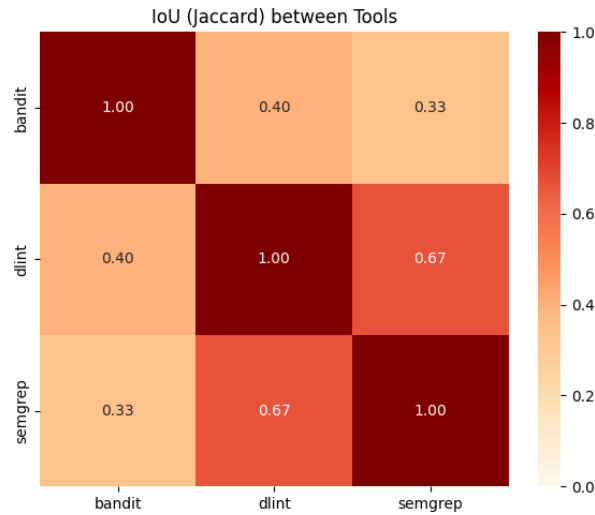


(b) dl-art-school: IoU



(c) two1-python: IoU

Figure 2: Per-project IoU (Jaccard) heatmaps: empty entries indicate no findings for that tool/project pair.



(a) IoU (Jaccard) heatmap between tools

Figure 3: Tool-level CWE coverage and IoU visualizations for the selected tools.

1.5.1 Interpreting the IoU Matrix

IoU (0–1) quantifies set overlap of CWE categories between tools. High IoU (e.g., ≥ 0.6) means the tools are largely redundant; semgrep vs dlint is the strongest overlap here (≈ 0.67). Moderate IoU (around 0.3–0.5) shows partial overlap and useful complementarity (bandit vs dlint ≈ 0.40 ; bandit vs semgrep ≈ 0.33). Low IoU indicates tool-specific coverage that increases the union when combined. Empty per-project heatmap cells mean the tool reported no CWE-tagged findings for that project.

Use IoU to pick pairs that balance non-redundancy with meaningful Top-25 contribution.

1.5.2 Which tool combination maximizes CWE coverage?

Heuristics: compare union sizes ($\|S_a \cup S_b\|$) and marginal Top-25 gains when adding a tool.

Summary for this dataset:

1. Best single (Top-25 oriented): semgrep or dlint (both have high fraction of their CWEs in Top-25).
2. Best pair: semgrep + bandit; semgrep is broad, bandit adds complementary categories (e.g., CWE-338 / CWE-428).
3. Best trio: semgrep + bandit + dlint; yields the largest union and therefore the highest overall coverage, with diminishing marginal returns from dlint.

Recommendation: run semgrep + bandit for a cost-effective, high-coverage baseline; add dlint when you need extra recall or confirmation for command-injection/shell issues. Recompute IoU after changing rules/projects or tool versions.

1.6 References

- Lecture 6 slides: [here](#)
- Lab Assignment 6 document: [here](#)
- SAST paper (FSE 2023): <https://sen-chen.github.io/img.cs/pdf/fse2023-sast.pdf>
- Static Analysis Tools List: <https://github.com/analysis-tools-dev/static-analysis>
- Jaccard Index (Wikipedia): https://en.wikipedia.org/wiki/Jaccard_index

2 Lab Assignment 7: Reaching Definitions Analyzer for C Programs

Repository Link(s): lab7/ (runner: lab7/run_analysis.py, code: lab7/analyzer/)

2.1 Introduction, Setup, and Tools

2.1.1 Introduction

I implemented an intraprocedural analyzer that constructed Control Flow Graphs (CFGs) for single-file C programs and ran Reaching Definitions analysis on top of them. I created DOT/PNG CFGs, per-iteration data-flow tables, a definitions index, an ambiguity report, and basic metrics including cyclomatic complexity.

2.1.2 Environment and Tools

- OS: Windows 11; Terminal: PowerShell 7
- Python: 3.13.7
- Required tools: Graphviz (for dot), Matplotlib (for plots)
- Editor: VS Code Insiders

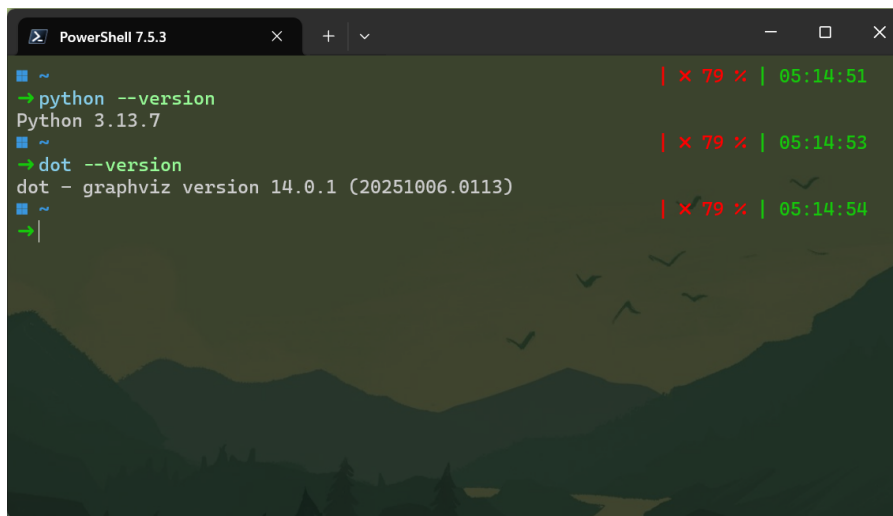


Figure 4: Environment Details

2.2 Program Corpus

I selected three single-file C programs (200–300 LOC each) that had conditionals, loops, and multiple reassignments, as required. I chose programs that were self-contained to avoid interprocedural complexity.

- Program 1: `inventory_tracker.c` – tracked stock levels with multiple conditionals and loop-based updates; good mix of assignments.
- Program 2: `student_performance.c` – computed aggregates with branch-heavy logic around grading thresholds.
- Program 3: `weather_simulator.c` – iterative simulation with nested condition checks and loop-carried updates.

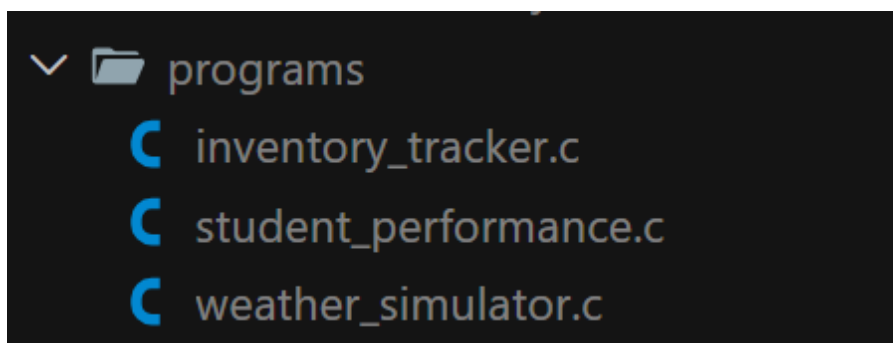


Figure 5: lab7/programs folder snapshot

2.3 Approach and Implementation

2.3.1 Parsing and Cleaning – analyzer/c_parser.py

- I first read the source and stripped preprocessor lines and comments: Removed `#...` lines and both `/* ... */` and `// ...` comments.

```
def read_c_file(path: str) -> str:
    with open(path, "r", encoding="utf-8") as handle:
        return handle.read()

def strip_preprocessor_and_comments(source: str) -> str:
    import re

    # Remove /* ... */ block comments
    source = re.sub(r"/\*.*?\*/", "", source, flags=re.S)
    # Remove // line comments
    source = re.sub(r"//.*", "", source)
    # Remove preprocessor lines
    lines = []
    for line in source.splitlines():
        if line.lstrip().startswith("#"):
            lines.append("")
        else:
            lines.append(line)
    return "\n".join(lines)
```

Figure 6: Code snippet – `strip_preprocessor_and_comments`

- Extracted only the body of `main()` and tracked its starting line so later line numbers stayed meaningful.

```
def find_main_function_bounds(source: str) -> Tuple[int, int]:
    import re

    # Find 'main' function signature
    match = re.search(r"\bint\s+main\s*\([^)]*\)\s*\{", source)
    if not match:
        # Try without assuming return type strictly
        match = re.search(r"\bmain\s*\([^)]*\)\s*\{", source)
    if not match:
        raise ValueError("No function named 'main' found in the provided file.")
    open_brace_idx = source.find("{", match.end() - 1)
    if open_brace_idx == -1:
        raise ValueError("Malformed main(): missing opening '{'.")
    depth = 0
    i = open_brace_idx
    while i < len(source):
        ch = source[i]
        if ch == '{':
            depth += 1
        elif ch == '}':
            depth -= 1
            if depth == 0:
                # body is between open_brace_idx+1 and i (exclusive)
                return (open_brace_idx + 1, i)
        i += 1
    raise ValueError("Malformed main(): missing closing '}'.")

def extract_main_function_source(source: str) -> Tuple[str, int]:
    start, end = find_main_function_bounds(source)
    body = source[start:end]
    start_line = source[:start].count("\n") + 1
    return body, start_line
```

Figure 7: Code snippet – `extract_main_function_source`

- I kept it deliberately simple; this worked fine for my inputs but wasn't a full C parser.

2.3.2 Basic Blocks and CFG Construction – analyzer/cfg_builder.py

I used leader-based basic block detection per the instructions given in the lab document. Blocks were labeled B0, B1, ... and I added edges for sequential flow, conditionals, and simple loops:

- Leaders: first statement, the condition of if/else if/else/while/for, the statement after a branch/jump.

```
# ----- Leader-based helpers -----
_COND_RE = re.compile(r'^\s*(if|else\s*if|else|while|for)\b')
_JUMP_RE = re.compile(r'^\s*(return|break|continue|goto)\b')

def _is_cond_or_loop(self, line: str) -> bool:
    return bool(self._COND_RE.match(line))

def _is_jump(self, line: str) -> bool:
    return bool(self._JUMP_RE.match(line))

def _find_leaders(self, lines: List[Tuple[str, int]]) -> List[int]:
    leaders: Set[int] = set()
    if lines:
        leaders.add(0)
    n = len(lines)
    for i, (txt, _) in enumerate(lines):
        if self._is_cond_or_loop(txt):
            leaders.add(i)
            if i + 1 < n:
                leaders.add(i + 1)
        if self._is_jump(txt):
            if i + 1 < n:
                leaders.add(i + 1)
        if txt.startswith('else'):
            leaders.add(i)
    return sorted(leaders)
```

Figure 8: Code snippet – `_find_leaders`

- Edges: for if I added true/false; for while/for I added back-edges from body to condition.
- Jumps like return/break/continue terminated flow for that block.

```
def _build_edges_from_blocks(self, lines: List[Tuple[str, int]], blocks: List[Tuple[str, int, List[Tuple[str, int]]]]) -> List[Tuple[str, str, str]]:
    edges: List[Tuple[str, str, str]] = []
    n = len(blocks)
    for i, (bname, _start_idx, blines) in enumerate(blocks):
        if not blines:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", ""))
            continue
        first = blines[0][0]
        last = blines[-1][0]
        cond = self._is_cond_or_loop(first)
        jump = self._is_jump(last)
        if cond:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", "true"))
            if i + 2 < n:
                edges.append((bname, f"B{i+2}", "false"))
            if first.startswith('while') or first.startswith('for'):
                if i + 1 < n:
                    edges.append((f"B{i+1}", bname, "back"))
        elif not jump:
            if i + 1 < n:
                edges.append((bname, f"B{i+1}", ""))
    # de-duplicate
    seen: Set[Tuple[str, str, str]] = set()
    uniq: List[Tuple[str, str, str]] = []
    for e in edges:
        if e not in seen:
            uniq.append(e)
            seen.add(e)
    return uniq
```

Figure 9: Code snippet – `_build_edges_from_blocks`

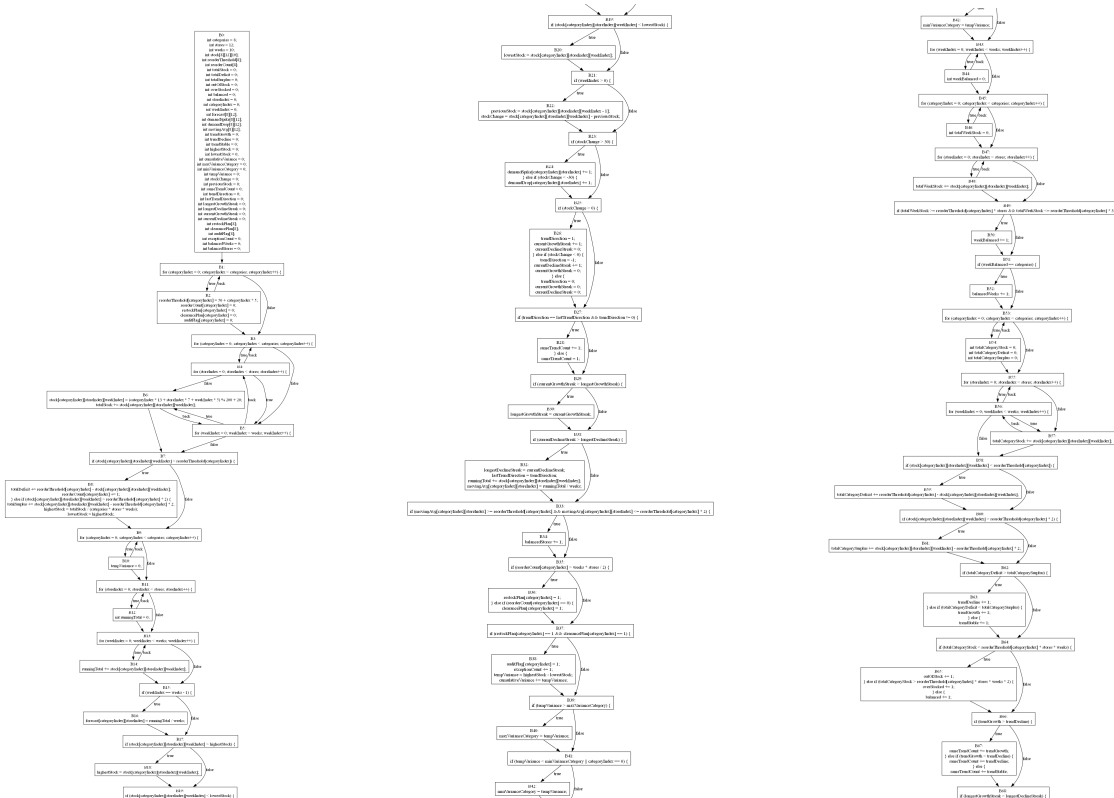
I exported a DOT graph for each program, and rendered it to PNG if dot was in PATH using Graphviz. Following in the code snippet shows the export and rendering logic.

```
def write_cfg_dot(cfg: CFG, destination: Path) -> None:
    lines: List[str] = ["digraph CFG {", "    node [shape=box];"]
    for block in cfg.blocks:
        label_lines = [f"{block.identifier}:"]
        for statement in block.statements:
            label_lines.append(statement.text)
        escaped = "\n".join(label_lines).replace("\n", "\\n")
        lines.append(f"    {block.identifier} [label=\"{escaped}\"];")
    for block in cfg.blocks:
        for edge in block.successors:
            if edge.label:
                lines.append(
                    f"    {block.identifier} -> {edge.target} [label=\"{edge.label}\"];")
            else:
                lines.append(f"    {block.identifier} -> {edge.target};")
    lines.append("}")
    destination.write_text("\n".join(lines), encoding="utf-8")

def render_dot_to_png(dot_file: Path, png_file: Optional[Path] = None) -> Optional[Path]:
    """Attempt to render a DOT file to PNG using Graphviz if available.

    Returns the path to the PNG file if successful, otherwise None.
    """
    if png_file is None:
        png_file = dot_file.with_suffix('.png')
    dot_executable = shutil.which("dot")
    if not dot_executable:
        print("Graphviz 'dot' executable not found in PATH. Skipping rendering.")
        return None
    try:
        subprocess.run([dot_executable, "-Tpng", str(dot_file), "-o", str(png_file)], check=True)
        return png_file
    except Exception:
        print(f"Failed to render {dot_file} to PNG.")
        return None
```

Figure 10: Code snippet – DOT export and PNG rendering



Link to sample CFG: [inventory_tracker CFG PNG](#) (too big to embed here)

2.3.3 Definition Extraction – analyzer/cfg_builder.py

For each statement, I extracted defined variables and created unique definition IDs (D1, D2, ...):

- Handled declarations like `int x = 0;`, `int x, y;` and simple assignments including compound ops like `+=`.
- Counted increments/decrements as definitions.
- Stored each definition with statement text, block ID, and line number.

I wrote a Markdown table (`definitions.md`) for quick reference. Following is the code snippet for the same:

```
# ----- Definitions extraction -----
def _record_definitions_from_text(self, statement_text: str, line: int, block_id: str) -> List[str]:
    vars_defined = self._extract_defined_variables_from_text(statement_text)
    ids: List[str] = []
    for var in vars_defined:
        did = self._new_definition_id()
        self.definitions[did] = Definition(identifier=did, variable=var, statement=statement_text.strip(),
                                           line=line, block_id=block_id)
        self._var_to_defs.setdefault(var, set()).add(did)
        ids.append(did)
    return ids

def _extract_defined_variables_from_text(self, text: str) -> Set[str]:
    s = text.strip().rstrip(';')
    vars_defined: Set[str] = set()
    # handle declarations like 'int x = 0;' or 'int x, y;'
    decl_match = re.match(r"(int|float|double|char|long|short|unsigned|signed)\b(.*)", s)
    if decl_match:
        rest = decl_match.group(2)
        # split by commas outside parentheses
        parts = [p.strip() for p in rest.split(',') if p.strip()]
        for p in parts:
            name = re.split(r"\s|=|["'"]", p.strip())
            if name and name[0]:
                vars_defined.add(name[0].lstrip('*'))
    # handle assignments like 'x = ...' and increments 'x++' '--x'
    assign = re.match(r"([A-Za-z_][A-Za-z0-9_\.]*)\s*([+|-|*|/|%&|^]?=)", s)
    if assign:
        vars_defined.add(assign.group(1))
    inc = re.match(r"([A-Za-z_][A-Za-z0-9_\.]*)\s*(\++|--)", s)
    pre_inc = re.match(r"(\++|--)\s*([A-Za-z_][A-Za-z0-9_\.]*)", s)
    if inc:
        vars_defined.add(inc.group(1))
    if pre_inc:
        vars_defined.add(pre_inc.group(2))
    return vars_defined
```

Figure 11: Code snippet – `_extract_defined_variables_from_text`

Definition ID	Variable	Block	Line	Statement
D1	categories	B0	2	<code>int categories = 8;</code>
D2	stores	B0	3	<code>int stores = 12;</code>
D3	weeks	B0	4	<code>int weeks = 10;</code>
D4	stock	B0	5	<code>int stock[8][12][10];</code>
D5	reorderThreshold	B0	6	<code>int reorderThreshold[8];</code>
D6	reorderCount	B0	7	<code>int reorderCount[8];</code>
D7	totalStock	B0	8	<code>int totalStock = 0;</code>
D8	totalDeficit	B0	9	<code>int totalDeficit = 0;</code>
D9	totalSurplus	B0	10	<code>int totalSurplus = 0;</code>
D10	outOfStock	B0	11	<code>int outOfStock = 0;</code>
D11	overStocked	B0	12	<code>int overStocked = 0;</code>
D12	balanced	B0	13	<code>int balanced = 0;</code>
D13	storeIndex	B0	14	<code>int storeIndex = 0;</code>
D14	categoryIndex	B0	15	<code>int categoryIndex = 0;</code>
D15	weekIndex	B0	16	<code>int weekIndex = 0;</code>
D16	forecast	B0	17	<code>int forecast[8][12];</code>
D17	demandSpike	B0	18	<code>int demandSpike[8][12];</code>
D18	demandDrop	B0	19	<code>int demandDrop[8][12];</code>
D19	movingAvg	B0	20	<code>int movingAvg[8][12];</code>
D20	trendGrowth	B0	21	<code>int trendGrowth = 0;</code>
D21	trendDecline	B0	22	<code>int trendDecline = 0;</code>
D22	trendStable	B0	23	<code>int trendStable = 0;</code>
D23	highestStock	B0	24	<code>int highestStock = 0;</code>
D24	lowestStock	B0	25	<code>int lowestStock = 0;</code>
D25	cumulativeVariance	B0	26	<code>int cumulativeVariance = 0;</code>
D26	maxVarianceCategory	B0	27	<code>int maxVarianceCategory = 0;</code>
D27	minVarianceCategory	B0	28	<code>int minVarianceCategory = 0;</code>
D28	tempVariance	B0	29	<code>int tempVariance = 0;</code>
D29	stockChange	B0	30	<code>int stockChange = 0;</code>
D30	previousStock	B0	31	<code>int previousStock = 0;</code>
D31	sameTrendCount	B0	32	<code>int sameTrendCount = 0;</code>

Figure 12: definitions.md preview

2.3.4 gen/kill Sets per Block – analyzer/cfg.builder.py

I computed `gen[B]` and `kill[B]` for each block:

- **gen[B] (generated definitions):** All definitions that are created inside block B.
- **kill[B] (killed definitions):** All other definitions of the same variables that exist outside block B.

I computed `gen[B]` and `kill[B]` as the latest definitions per variable within a block, in program order. The code snippet below shows the implementation (`compute_gen_kill_sets`).

```
def _compute_gen_kill_sets(self) -> None:
    for block in self._blocks:
        latest_by_var: Dict[str, str] = {}
        new_gen: List[str] = []
        for statement in block.statements:
            for definition_id in statement.definition_ids:
                definition = self._definitions[definition_id]
                previous = latest_by_var.get(definition.variable)
                if previous and previous in new_gen:
                    new_gen.remove(previous)
                latest_by_var[definition.variable] = definition_id
                new_gen.append(definition_id)
        block.gen = new_gen
        kills: Set[str] = set()
        for variable, definition_id in latest_by_var.items():
            all_defs = self._var_to_defs.get(variable, set())
            kills.update(all_defs - {definition_id})
        block.kill = kills
        block.in_set = set()
        block.out_set = set(block.gen)
```

Figure 13: Code snippet – `_compute_gen_kill_sets`

2.3.5 Reaching Definitions (Data-Flow) – analyzer/dataflow.py

I implemented the standard forward data-flow equations (analyzer/dataflow.py):

- $in[B] = \bigcup out[P]$ over predecessors P
- $out[B] = gen[B] \cup (in[B] - kill[B])$

I iterated until the sets stopped changing. I recorded every iteration as a snapshot and wrote a Markdown table with gen/kill/in/out for each block per iteration. Following is the snippet for the iteration logic:

```
def compute_reaching_definitions(cfg: CFG, max_iterations: int = 100) -> List[Dict[str, Dict[str, Set[str]]]]:
    block_map = cfg.block_map
    snapshots: List[Dict[str, Dict[str, Set[str]]]] = []
    initial_snapshot: Dict[str, Dict[str, Set[str]]] = {}
    for block in cfg.blocks:
        initial_snapshot[block.identifier] = {
            "gen": set(block.gen),
            "kill": set(block.kill),
            "in": set(block.in_set),
            "out": set(block.out_set),
        }
    snapshots.append(initial_snapshot)
    iteration = 0
    changed = True
    while changed and iteration < max_iterations:
        iteration += 1
        changed = False
        snapshot: Dict[str, Dict[str, Set[str]]] = {}
        for block in cfg.blocks:
            in_set: Set[str] = set()
            for predecessor_id in block.predecessors:
                predecessor = block_map[predecessor_id]
                in_set.update(predecessor.out_set)
            out_set = set(block.gen) | (in_set - block.kill)
            if in_set != block.in_set or out_set != block.out_set:
                block.in_set = in_set
                block.out_set = out_set
                changed = True
            snapshot[block.identifier] = {
                "gen": set(block.gen),
                "kill": set(block.kill),
                "in": set(block.in_set),
                "out": set(block.out_set),
            }
        snapshots.append(snapshot)
    return snapshots
```

Figure 14: Code snippet – compute_reaching_definitions

Iteration 0				
Basic Block	gen[B]	kill[B]	in[B]	out[B]
B0	{D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38, D39}	{D40, D41, D42, D43, D44, D45, D46, D47, D48, D49, D50, D51, D52, D53, D54, D60, D61, D62, D63, D64, D65, D66, D67}	{}	{D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23, D24, D25, D26, D27, D28, D29, D30, D31, D32, D33, D34, D35, D36, D37, D38, D39}
B1	{}	{}	{}	{}
B2	{}	{}	{}	{}
B3	{}	{}	{}	{}
B4	{}	{}	{}	{}
B5	{}	{}	{}	{}
B6	{}	{}	{}	{}
B7	{}	{}	{}	{}
B8	{}	{}	{}	{}
B9	{D40, D41}	{D14, D15, D42, D43}	{}	{D40, D41}
B10	{}	{}	{}	{}
B11	{D42, D43}	{D14, D15, D40, D41}	{}	{D42, D43}
B12	{}	{}	{}	{}
B13	{}	{}	{}	{}
B14	{}	{}	{}	{}
B15	{}	{}	{}	{}
B16	{}	{}	{}	{}
B17	{D44}	{D7, D64, D67}	{}	{D44}
B18	{}	{}	{}	{}
B19	{D45, D46}	{D8, D9, D62, D63, D66}	{}	{D45, D46}
B20	{}	{}	{}	{}
B21	{}	{}	{}	{}
B22	{}	{}	{}	{}
B23	{D47}	{D23}	{}	{D47}
B24	{}	{}	{}	{}
B25	{D48, D49, D50}	{D24, D33, D34}	{}	{D48, D49, D50}
B26	{}	{}	{}	{}
B27	{D51}	{D36, D61}	{}	{D51}

Figure 15: reaching_definitions_iterations.md preview for iteration 0

2.3.6 Ambiguity Detection – analyzer/dataflow.py

After convergence, I flagged blocks where a variable had more than one reaching definition in in[B] (`find_ambiguous_variables`). I summarized them with the corresponding statements for context (`ambiguity.md`).

```
def find_ambiguous_variables(cfg: CFG) -> Dict[str, Dict[str, Set[str]]]:
    ambiguous: Dict[str, Dict[str, Set[str]]] = {}
    for block in cfg.blocks:
        var_to_defs: Dict[str, Set[str]] = {}
        for definition_id in block.in_set:
            definition = cfg.definitions[definition_id]
            var_to_defs.setdefault(definition.variable, set()).add(definition_id)
        ambiguous_vars = {var: defs for var, defs in var_to_defs.items() if len(defs) > 1}
        if ambiguous_vars:
            ambiguous[block.identifier] = ambiguous_vars
    return ambiguous
```

Figure 16: Code snippet – `find_ambiguous_variables`

Block B3		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>
Block B4		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>
Block B5		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>
Block B6		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>
Block B7		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>
Block B8		
Variable	Definition IDs	Statements
totalStock	D7, D44	<pre>int totalStock = 0; totalStock += stock[categoryIndex][storeIndex][weekIndex];</pre>

Figure 17: `ambiguity.md` preview

2.3.7 Runner and Outputs

The runner (lab7/run_analysis.py) processed all .c files in lab7/programs/ and wrote per-program outputs to lab7/output/<program_name>/:

- cfg.dot (+ cfg.png if Graphviz was available)
- definitions.md
- reaching_definitions_iterations.md
- ambiguity.md
- summary.json (nodes, edges, CC)

```
def analyze_program(program_path: Path) -> Dict[str, int]:
    print(f"[info] Analyzing {program_path.name} ...")
    source = read_c_file(str(program_path))
    cleaned = strip_preprocessor_and_comments(source)
    body, start_line = extract_main_function_source(cleaned)

    cfg = build_cfg_from_main_source(body, start_line)

    snapshots = compute_reaching_definitions(cfg)
    ambiguous = find_ambiguous_variables(cfg)

    nodes = len(cfg.blocks)
    edges = sum(len(block.successors) for block in cfg.blocks)
    cyclomatic_complexity = edges - nodes + 2

    program_name = program_path.stem
    program_output_dir = OUTPUT_DIR / program_name
    program_output_dir.mkdir(parents=True, exist_ok=True)

    dot_path = program_output_dir / "cfg.dot"
    write_cfg_dot(cfg, dot_path)
    render_dot_to_png(dot_path)
    write_definitions_table(cfg.definitions, program_output_dir / "definitions.md")
    write_iteration_tables(snapshots, cfg, program_output_dir / "reaching_definitions_iterations.md")
    write_ambiguity_report(ambiguous, cfg.definitions, program_output_dir / "ambiguity.md")
    write_program_summary(
        program_name,
        {"nodes": nodes, "edges": edges, "cc": cyclomatic_complexity},
        program_output_dir / "summary.json",
    )

    print(f"[done] {program_name}: N={nodes}, E={edges}, CC={cyclomatic_complexity}")
    return {"program": program_name, "nodes": nodes, "edges": edges, "cc": cyclomatic_complexity}
```

Figure 18: Code snippet – Runner invoking analysis and writing outputs

```
~\Desktop\sem-5\CS 202\cs202-stt :: |main| +2 ~3 | 74 ~3
| 79 74 | 06:26:29
→ .venv\Scripts\activate
~\Desktop\sem-5\CS 202\cs202-stt :: |main| +2 ~3 | 74 ~3
| 79 74 | 06:26:40
→ python .\lab7\run_analysis.py
[info] Analyzing inventory_tracker.c ...
[done] inventory_tracker: N=76, E=126, CC=52
[info] Analyzing student_performance.c ...
[done] student_performance: N=59, E=98, CC=41
[info] Analyzing weather_simulator.c ...
[done] weather_simulator: N=76, E=120, CC=46
~\Desktop\sem-5\CS 202\cs202-stt :: |main| +2 ~3 | 74 ~3
| 79 74 | 06:27:02
→ |
```

Figure 19: Terminal Output

I also wrote code to generate an overall metrics_summary.md and, if Matplotlib was available, a bar chart cyclomatic_complexity.png at lab7/output/.

```
def generate_metrics_summary(metrics: List[Dict[str, int]]) -> None:
    rows = [
        " | Program | Nodes (N) | Edges (E) | Cyclomatic Complexity (CC) |",
        " |-----|-----|-----|-----|",
    ]
    for entry in metrics:
        rows.append(
            f" | {entry['program']} | {entry['nodes']} | {entry['edges']} | {entry['cc']} |"
        )
    (OUTPUT_DIR / "metrics_summary.md").write_text("\n".join(rows), encoding="utf-8")
    plot_cyclomatic_complexity(metrics, OUTPUT_DIR / "cyclomatic_complexity.png")
```

Figure 20: Code snippet – Metrics summary generation

Program	Nodes (N)	Edges (E)	Cyclomatic Complexity (CC)
inventory_tracker	76	126	52
student_performance	59	98	41
weather_simulator	76	120	46

Figure 21: metrics_summary.md preview

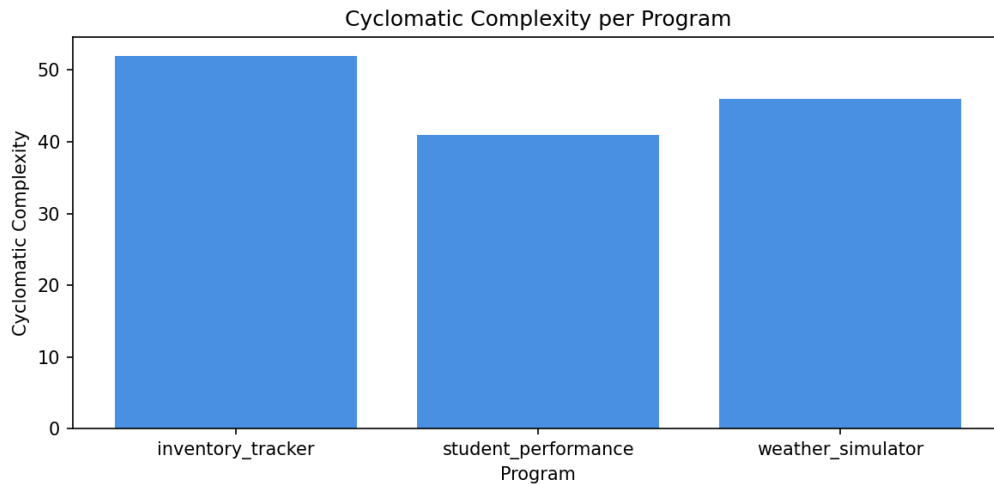


Figure 22: Cyclomatic Complexity Chart

2.4 Results

I am attaching links to key outputs for each program below because they are too large to embed here.

2.4.1 CFG and Metrics

Links to CFG PNGs for each program:

- [inventory_tracker](#)
- [student_performance](#)
- [weather_simulator](#)

For each program, I reported the number of nodes (basic blocks), edges, and cyclomatic complexity $CC = E - N + 2$. The chart helped me compare CCs across the three programs at a glance.

Actual metrics from my run ([lab7/output/metrics_summary.md](#)):

Program	Nodes (N)	Edges (E)	Cyclomatic Complexity (CC)
inventory_tracker	76	126	52
student_performance	59	98	41
weather_simulator	76	120	46

2.4.2 Reaching Definitions Tables

I skimmed early iterations to sanity-check propagation, and I confirmed convergence when tables stopped changing. I then used the final snapshot to interpret variable reachability.

Links to final iteration tables for each program:

- [inventory_tracker](#)
- [student_performance](#)
- [weather_simulator](#)

2.4.3 Ambiguity Highlights

I listed a few examples where variables had multiple reaching definitions at a program point and explained why (e.g., different branches or loop-carried definitions).

Links to detailed ambiguity tables for each program:

- [inventory_tracker](#)
- [student_performance](#)
- [weather_simulator](#)

2.5 Discussion and Reflection

2.5.1 Challenges

I faced a few challenges while implementing the analyzer. Getting the leader-based heuristics to behave well across nested conditionals, chained `else if` constructs, and loops required careful compromises. I deliberately kept the reconstruction simple, which worked for my programs but isn't a full CFG reconstructor. I also had to filter out syntactic noise early (like standalone braces and empty statements) to keep basic blocks clean. Finally, I ensured that Graphviz was installed and on PATH by updating the environment variables on my Windows machine.

2.5.2 What I Learned

This assignment gave me a hands-on understanding of how control-flow and data-flow analyses fit together in practice. Building CFGs forced me to think in terms of basic blocks, leaders, and edges for branches and loops. I got a better understanding of data-flow analysis by defining `gen/kill` precisely and then iterating the reaching definitions equations $in[B] = \bigcup out[P]$ and $out[B] = gen[B] \cup (in[B] - kill[B])$. Tracing iteration snapshots helped me verify convergence and understand why certain variables had multiple reaching definitions at a point (ambiguity due to branches or loop-carried updates). Overall, I learned how to move from raw C source to CFGs, to `gen/kill` sets, to stable reaching definitions.

2.5.3 Further Improvements

- Swap the lightweight parsing for a proper C front-end (e.g., `pycparser`) to cover more syntax and nesting reliably.
- Refine CFG edges for more complex constructs and labels.
- Add unit tests for tricky statements (declarations with pointers/arrays, compound statements).

2.6 References

- Lecture 7 slides: [here](#)
- Lab Assignment 7 document: [here](#)
- Data-flow analysis: https://en.wikipedia.org/wiki/Data-flow_analysis
- Graphviz: <https://graphviz.org/>
- PyGraphviz: <https://pygraphviz.github.io/>